

Go by Example: Atomic Counters

The primary mechanism for managing state in Go is communication over channels. We saw this for example with [worker pools](#). There are a few other options for managing state though. Here we'll look at using the `sync/atomic` package for *atomic counters* accessed by multiple goroutines.

We'll use an atomic integer type to represent our (always-positive) counter.

A `WaitGroup` will help us wait for all goroutines to finish their work.

We'll start 50 goroutines that each increment the counter exactly 1000 times.

To atomically increment the counter we use `Add`.

Wait until all the goroutines are done.

Here no goroutines are writing to 'ops', but using `Load` it's safe to atomically read a value even while other goroutines are (atomically) updating it.

We expect to get exactly 50,000 operations. Had we used a non-atomic integer and incremented it with `ops++`, we'd likely get a different number, changing between runs, because the goroutines would interfere with each other. Moreover, we'd get data race failures when running with the `-race` flag.

Next we'll look at mutexes, another tool for managing state.

Next example: [Mutexes](#).

by [Mark McGranaghan](#) and [Eli Bendersky](#) | [source](#) | [license](#)

```
package main

import (
    "fmt"
    "sync"
    "sync/atomic"
)

func main() {
    var ops atomic.Uint64

    var wg sync.WaitGroup

    for range 50 {
        wg.Go(func() {
            for range 1000 {
                ops.Add(1)
            }
        })
    }

    wg.Wait()

    fmt.Println("ops:", ops.Load())
}
```

```
$ go run atomic-counters.go
ops: 50000
```